

Machine Learning and Artificial Intelligence

Lecturer: Do Ngoc Hung

Email: dnhung@hcmiu.edu.vn

Textbook:

1. Simon J.D. Prince, "Understanding Deep Learning"
2. C.M. Bishop, "Pattern Recognition and Machine Learning"
3. I. Goodfellow, Y. Bengio, A. Courville, "Deep learning"
4. M. Paluszczek, S. Thomas, "Matlab Machine Learning"

Lecture Note: Blackboard

Programming Language: Matlab, Python

Grading Criteria

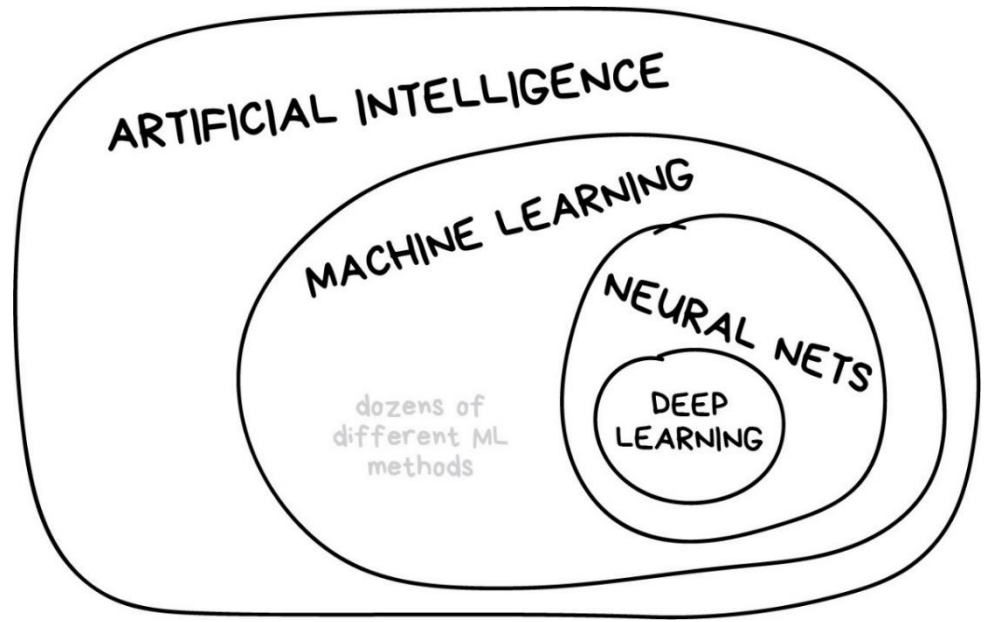
Attendance + Quiz + HW + Project: 30%

Midterm Examination: 20%

Final Examination: 50%

What is Artificial Intelligence?

The term "artificial intelligence" is often used to describe machines (or computers) that mimic "cognitive" functions that humans associate with the human mind, such as "learning" and "problem solving"



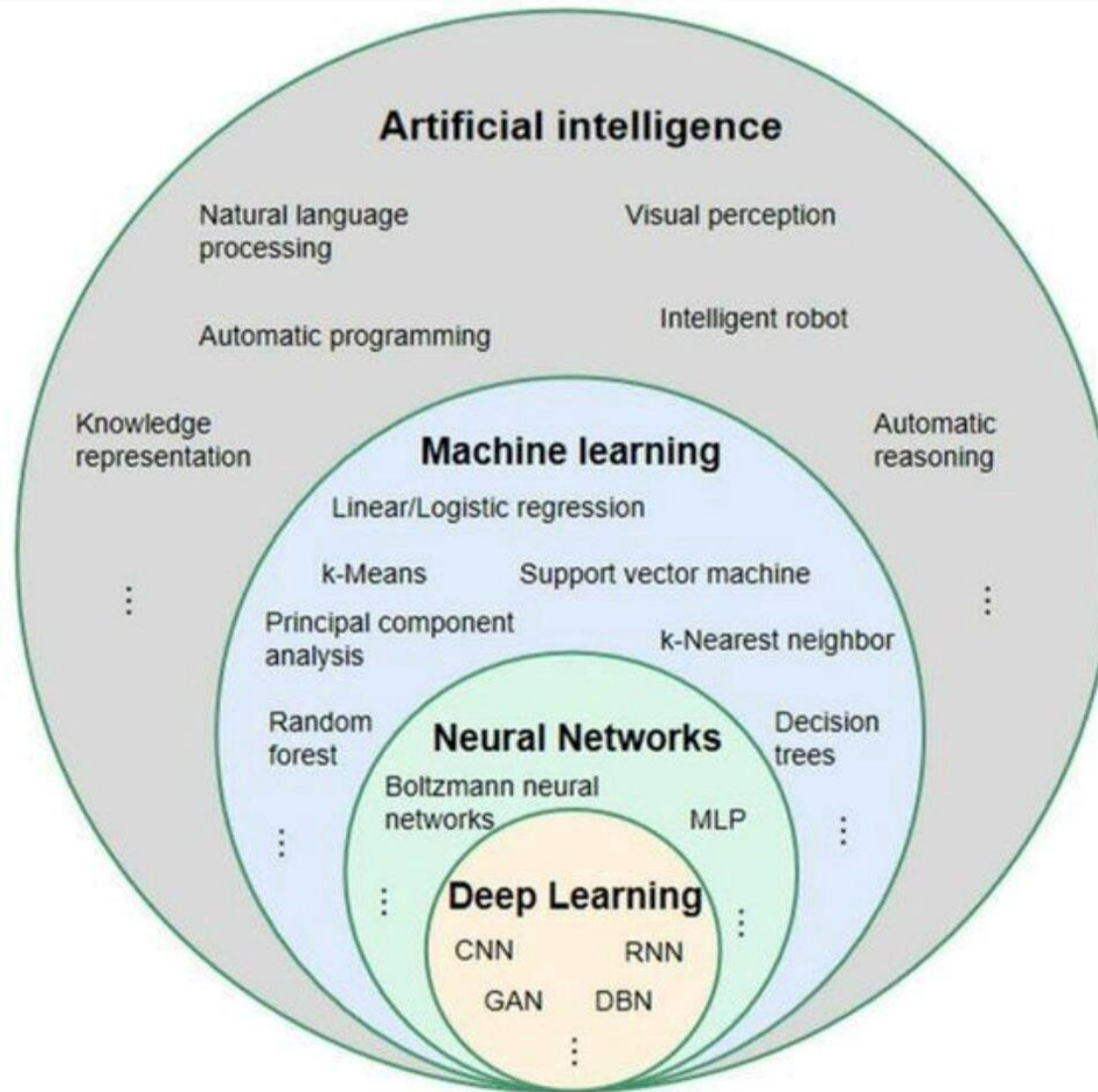
Artificial intelligence is the name of a whole knowledge field, similar to biology or chemistry.

Machine Learning is a part of artificial intelligence. An important part, but not the only one.

Neural Networks are one of machine learning types. A popular one, but there are other good guys in the class.

Deep Learning is a modern method of building, training, and using neural networks.

Relationship between artificial intelligence, machine learning, neural network, and deep learning



What is Machine Learning?

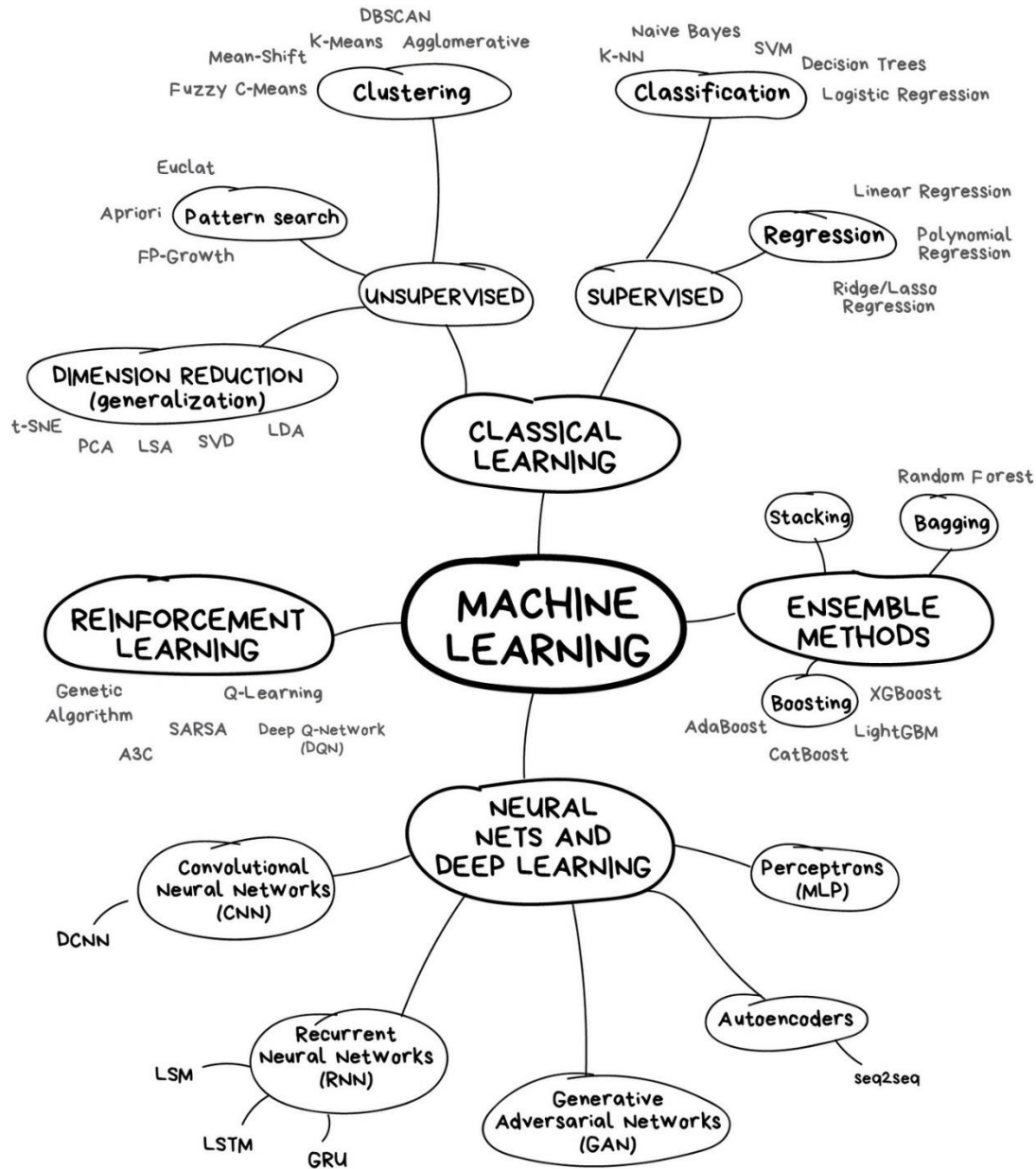
- It is very hard to write programs that solve problems like recognizing a face.
 - We don't know what program to write because we don't know how our brain does it.
 - Even if we had a good idea about how to do it, the program might be horrendously complicated.
- Instead of writing a program by hand, we collect lots of examples that specify the correct output for a given input.
- A machine learning algorithm then takes these examples and produces a program that does the job.
 - The program produced by the learning algorithm may look very different from a typical hand-written program. It may contain millions of numbers.
 - If we do it right, the program works for new cases as well as the ones we trained it on.
- Massive amounts of computation are now cheaper than paying someone to write a task-specific program.

Some more examples of tasks that are best solved by using a learning algorithm

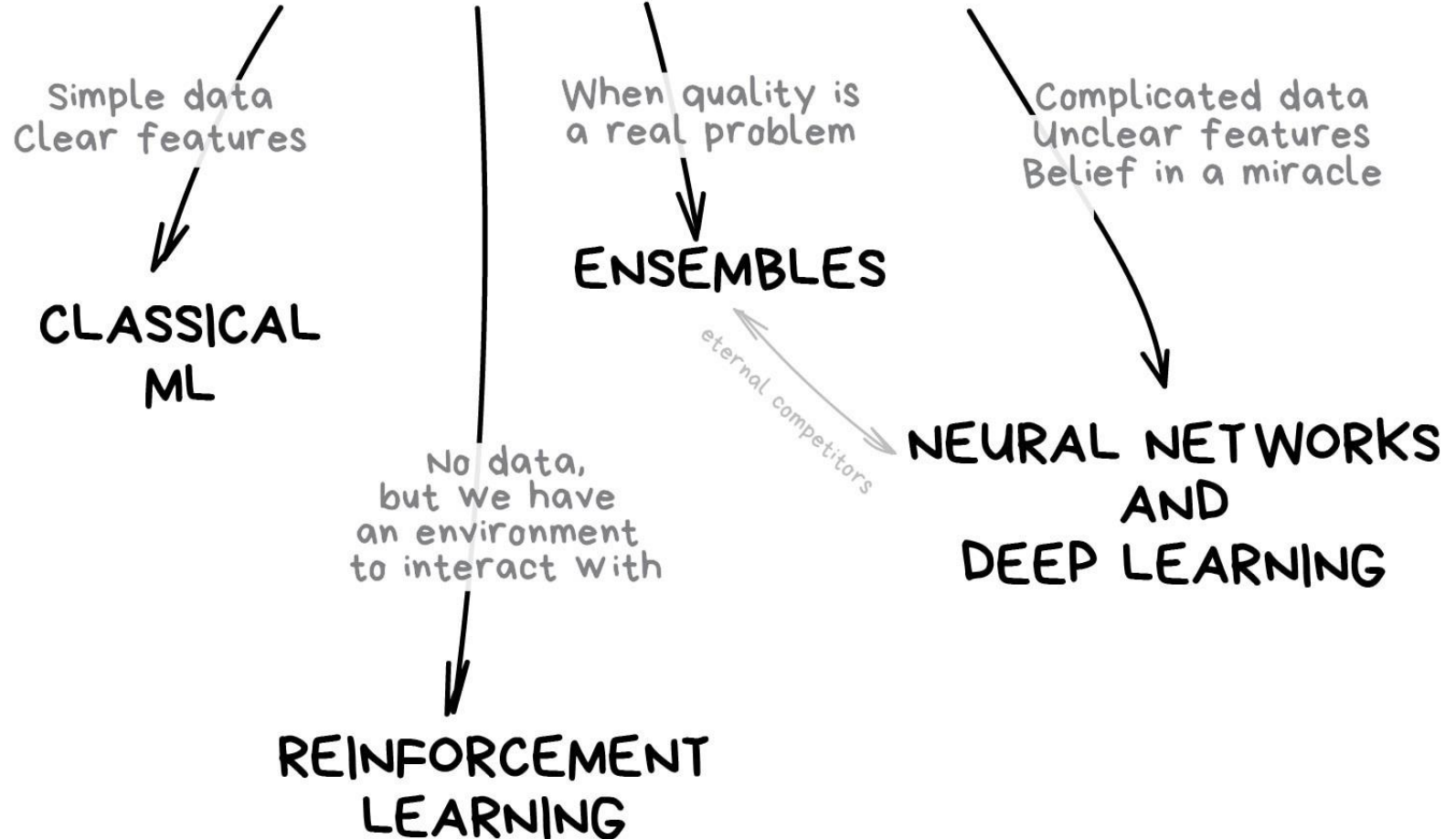
- Recognizing patterns:
 - Facial identities or facial expressions
 - Handwritten or spoken words
 - Medical images
- Generating patterns:
 - Generating images or motion sequences
- Recognizing anomalies:
 - Unusual sequences of credit card transactions
 - Unusual patterns of sensor readings in a nuclear power plant or unusual sound in your car engine.
- Prediction:
 - Future stock prices or currency exchange rates
 - Which movies will a person like?

Some web-based examples of machine learning

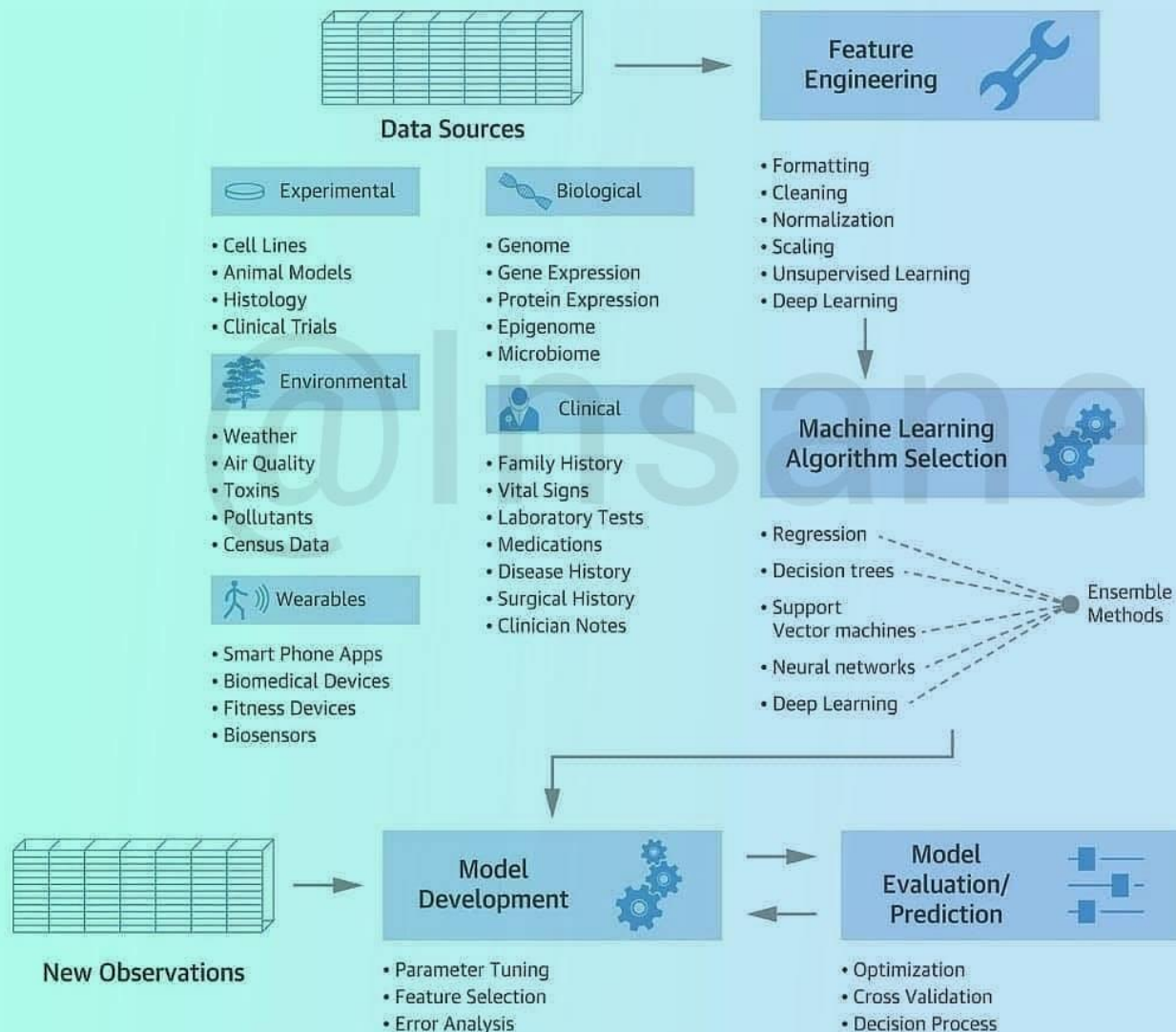
- The web contains a lot of data. Tasks with very big datasets often use machine learning
 - especially if the data is noisy or non-stationary.
- Spam filtering, fraud detection:
 - The enemy adapts so we must adapt too.
- Recommendation systems:
 - Lots of noisy data. Million dollar prize!
- Information retrieval:
 - Find documents or images with similar content.
- Data Visualization:
 - Display a huge database in a revealing way



THE MAIN TYPES OF MACHINE LEARNING



Machine Learning Workflow



Identify the problem to be solved and create a clear objective.

Define objective

Preparing data is a crucial step and involves building workflows to clean, match and blend the data.

Collect Data

Data is fed as input and the algorithm configured with the required parameters. A percent of the data can be utilized to train the model.

Prepare Data

Select Algorithm

Train Model

Publish the prepared experiment as a web service, so applications can use the model

Test Model

Integrate Model

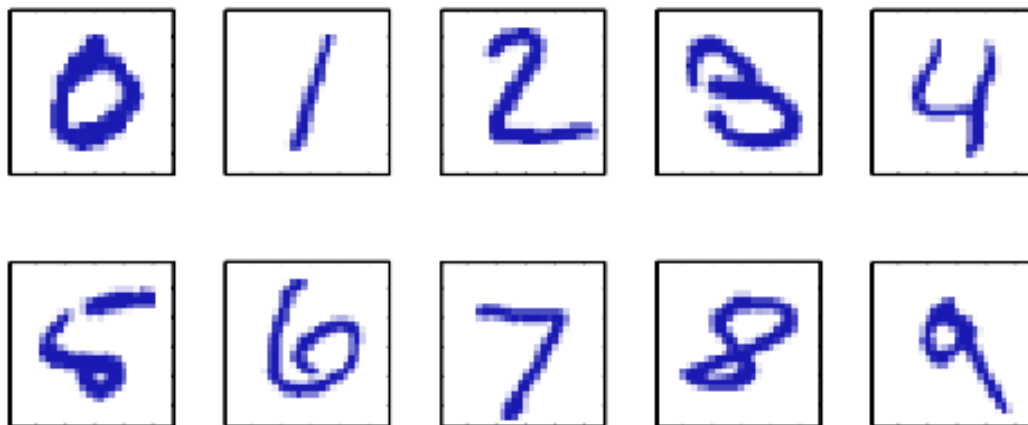
Collect data from hospitals, health insurance companies, social service agencies, police and fire dept.

Depending on the problem to be solved and the type of data, an appropriate algorithm will be chosen.

The remaining data is utilized to test the model for accuracy. Depending on the results, improvements can be performed in the "Train model" and/or "Select Algorithm" phases, iteratively.

Examples of AI Application

Hand written digits recognition



Each digit corresponds to a 28×28 pixel image and so can be represented by a vector $\mathbf{x}$ comprising 784 real numbers

The goal is to build a machine that will take such a vector $\mathbf{x}$ as input and that will produce the identity of the digit 0,...,9 as the output

Hand written digits recognition

Machine learning approach

Training set: a large set of N digits $\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$

Target vector $\mathbf{t}$: the categories of the digits in the training set are known in advance

Result $\mathbf{y}(\mathbf{x})$: takes a new digit image $\mathbf{x}$ as input and generates an output vector $\mathbf{y}$

The precise of $\mathbf{y}(\mathbf{x})$ is determined during the **training phase** (**learning phase**) on the basis of the training data

The ability to categorize correctly new examples that differ from those used for training is known as **generalization**

A classic example of a task that requires machine learning: It is very hard to say what makes a 2

0 0 0 1 1 1 1 1 1 2

2 2 2 2 2 2 2 3 3 3

3 4 4 4 4 4 5 5 5 5

6 6 7 7 7 7 7 8 8 8

9 9 9 9 9 9 9 9 9

A standard example of machine learning

- A lot of genetics is done on fruit flies.
 - They are convenient because they breed fast.
 - We already know a lot about them.
- The MNIST database of hand-written digits is the the machine learning equivalent of fruit flies.
 - They are publicly available and we can learn them quite fast in a moderate-sized neural net.
 - We know a huge amount about how well various machine learning methods do on MNIST.
- We usually use MNIST as our standard task.

Beyond MNIST: The ImageNet task

- 1000 different object classes in 1.3 million high-resolution training images from the web.
 - Best system in 2010 competition got 47% error for its first choice and 25% error for its top 5 choices.
- Jitendra Malik (an eminent neural net sceptic) said that this competition is a good test of whether deep neural networks work well for object recognition.
 - A very deep neural net (Krizhevsky et. al. 2012) gets less than 40% error for its first choice and less than 20% for its top 5 choices.

The Speech Recognition Task

- A speech recognition system has several stages:
 - **Pre-processing**: Convert the sound wave into a vector of acoustic coefficients. Extract a new vector about every 10 mil seconds.
 - **The acoustic model**: Use a few adjacent vectors of acoustic coefficients to place bets on which part of which phoneme is being spoken.
 - **Decoding**: Find the sequence of bets that does the best job of fitting the acoustic data and also fitting a model of the kinds of things people say.
- Deep neural networks pioneered by George Dahl and Abdel-rahman Mohamed are now replacing the previous machine learning method for the acoustic model.

Machine Learning & Symbolic AI

- **Knowledge Representation** works with facts/assertions and develops rules of logical inference. The rules can handle quantifiers. Learning and uncertainty are usually ignored.
- **Expert Systems** used logical rules or conditional probabilities provided by “experts” for specific domains.
- **Graphical Models** treat uncertainty properly and allow learning (but they often ignore quantifiers and use a fixed set of variables)
 - **Set of logical assertions** → values of a subset of the variables and local models of the probabilistic interactions between variables.
 - **Logical inference** → probability distributions over subsets of the unobserved variables (or individual ones)
 - Learning = refining the local models of the interactions.

Machine Learning & Statistics

- A lot of machine learning is just a rediscovery of things that statisticians already knew. This is often disguised by differences in terminology:
 - Ridge regression = weight-decay
 - Fitting = learning
 - Held-out data = test data
- But the emphasis is very different:
 - A good piece of statistics: Clever proof that a relatively simple estimation procedure is asymptotically unbiased.
 - A good piece of machine learning: Demonstration that a complicated algorithm produces impressive results on a specific task.
- Data-mining: Using very simple machine learning techniques on very large databases because computers are too slow to do anything more interesting with ten billion examples.

A spectrum of machine learning tasks

Statistics-----Artificial Intelligence

- Low-dimensional data (e.g. less than 100 dimensions)
 - Lots of noise in the data
 - There is not much structure in the data, and what structure there is, can be represented by a fairly simple model.
 - The main problem is distinguishing true structure from noise.
- High-dimensional data (e.g. more than 100 dimensions)
 - The noise is not sufficient to obscure the structure in the data if we process it right.
 - There is a huge amount of structure in the data, but the structure is too complicated to be represented by a simple model.
 - The main problem is figuring out a way to represent the complicated structure that allows it to be learned.

Types of learning task

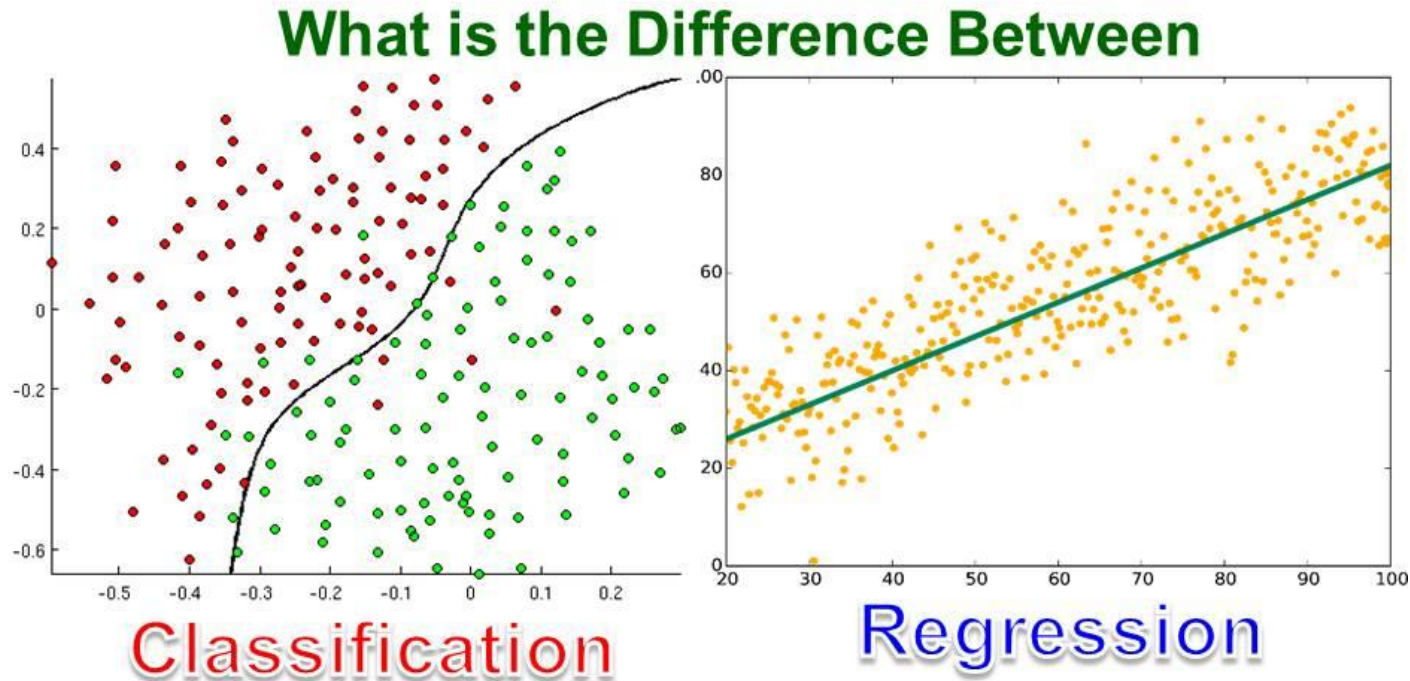
- Supervised learning
 - Learn to predict output when given an input vector
 - **Classification:** assign each input vector to one of the finite number of categories

When there are only two classes, then the technique is known as binary classification and the algorithm is called a binary classifier
 - **Regression:** Desired output consists of one or several continuous variables

In the case of regression the objective is to find the a relationship among the input variables. Regression analysis helps in understanding how the dependent variable changes with respect to the independent variables.

Types of learning task

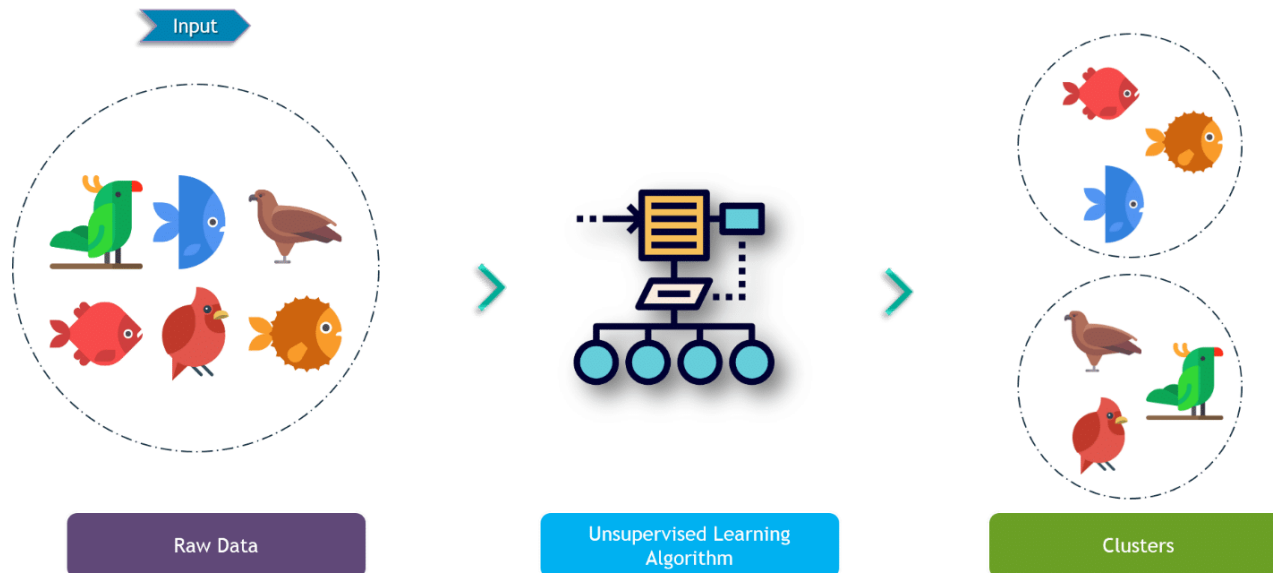
- Supervised learning



In case of linear regression, the objective is to fit a straight line through the data points to obtain the best fit.

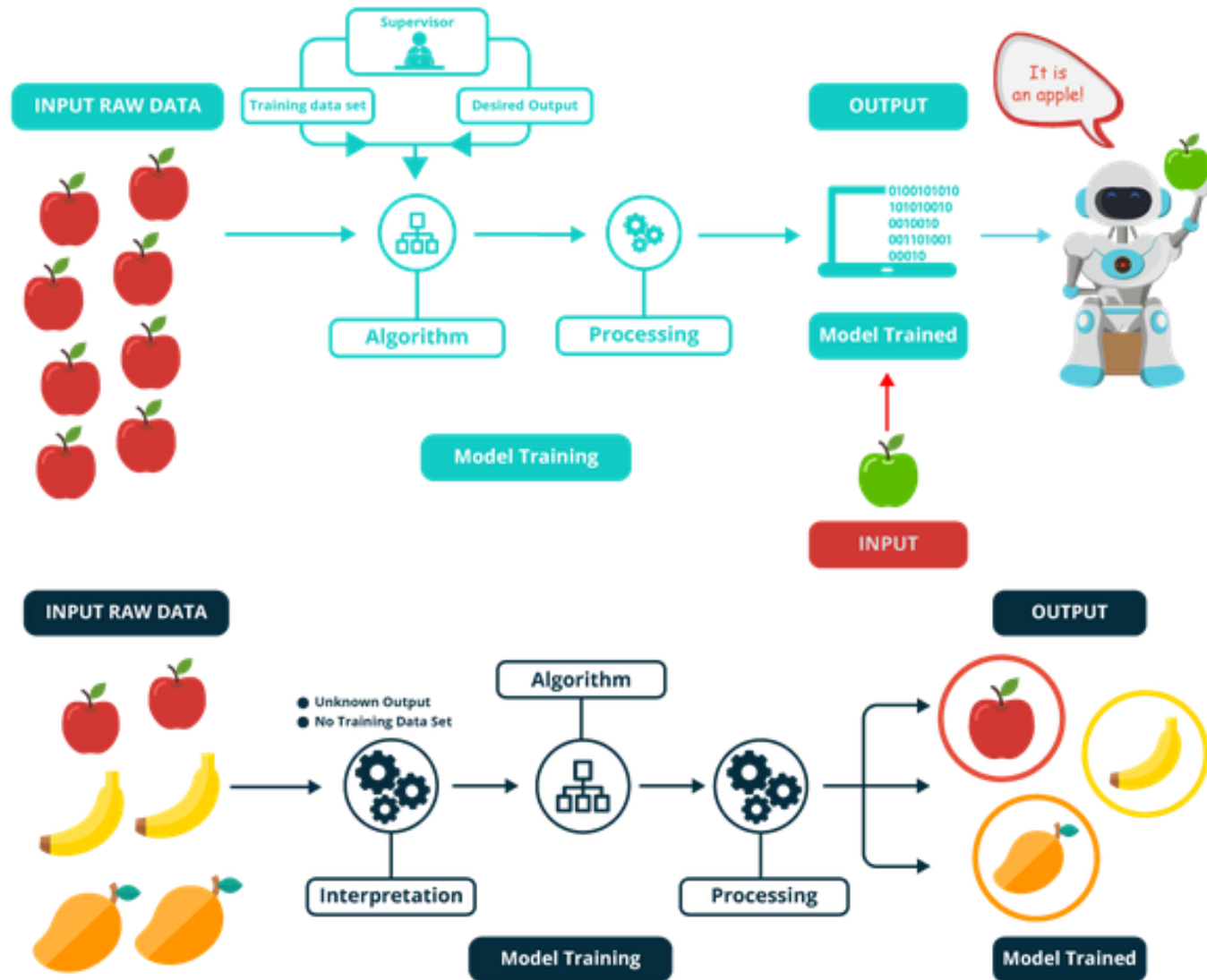
Types of learning task

- Unsupervised learning
 - Create an internal representation of the input e.g. form clusters; extract features
 - This is the new frontier of machine learning because most big datasets do not come with labels.
 - The algorithms available for the unsupervised learning are: Clusters, Anomaly detection, Association discovery, Training Models



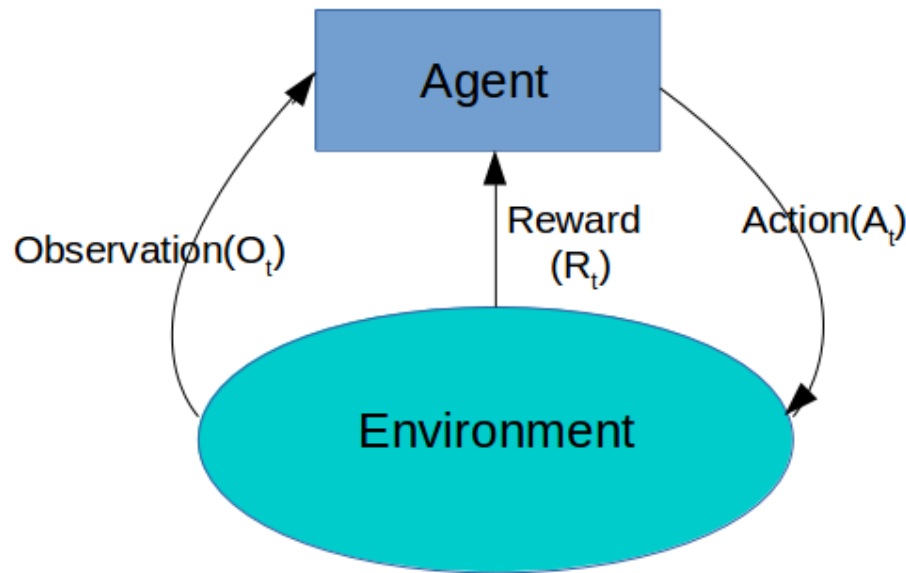
Types of learning task

- Supervised learning vs Unsupervised learning



Types of learning task

- Reinforcement learning
 - Learn action to maximize a reward
 - Reinforcement Learning is the science of making optimal decisions.

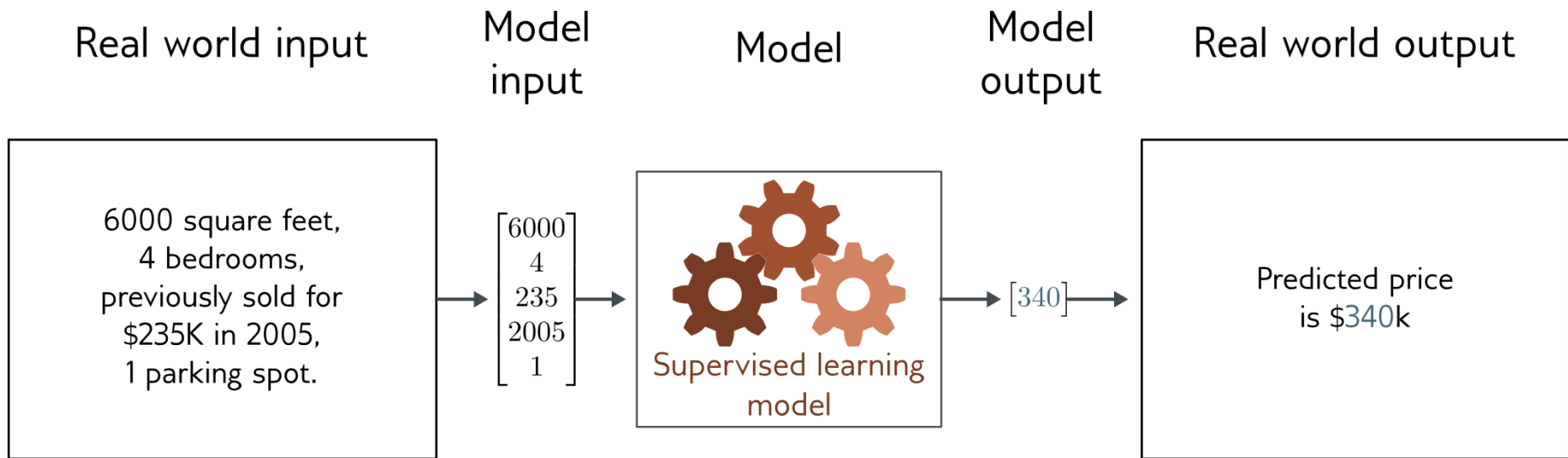


Types of learning task

- Reinforcement learning
 - There is no supervisor present. So, there is no one to tell you the best possible action. You just get a reward for each action.
 - Importance of time. Reinforcement Learning pays much attention to sequential data unlike other models where you receive random inputs. Here the input at the next step will always be dependent on input at previous state.
 - Concept of delayed rewards. You may not get a reward at each step. A reward may be given only after the completion of the entire task. Also, say you get a reward at a step only to find out that you made a huge blunder in a future step.
 - The agents action effects its next input. Say, you have the choice of going either left or right. After you take the action, the input at the next time step will be different if you chose right rather than left.

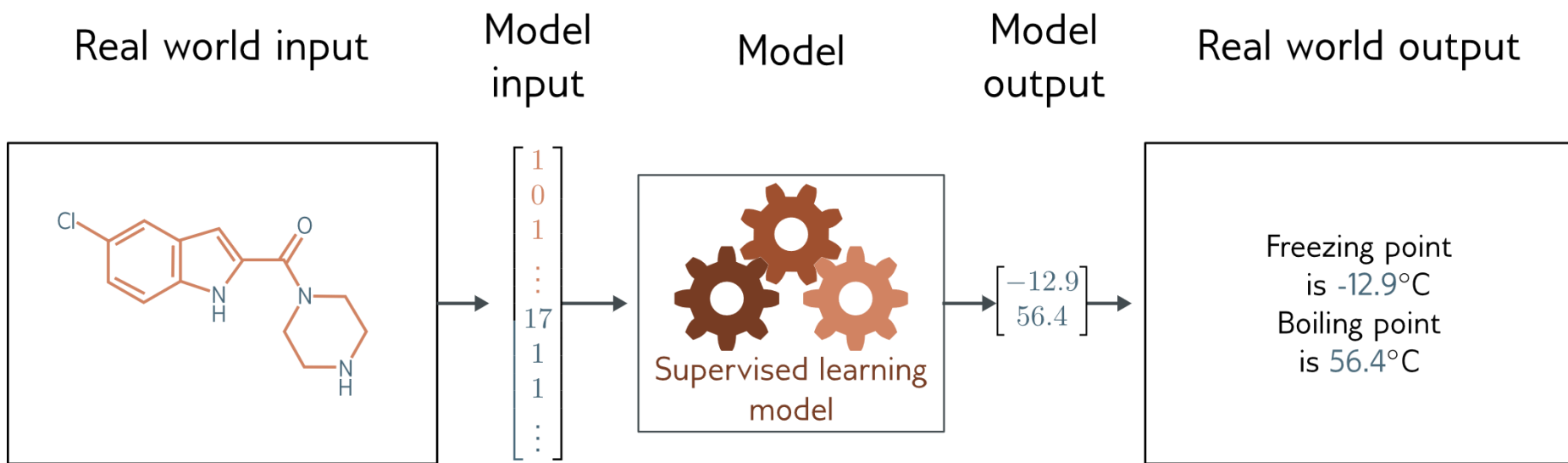
Five simple examples

Regression



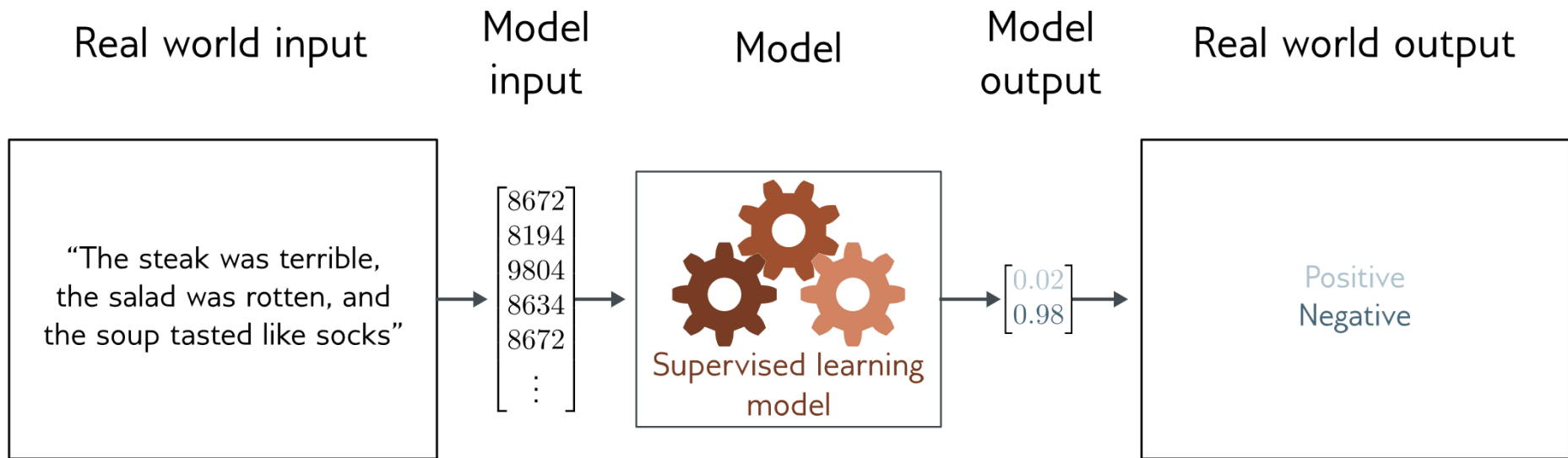
- Univariate regression problem (one output, real value)
- Fully connected network

Graph regression



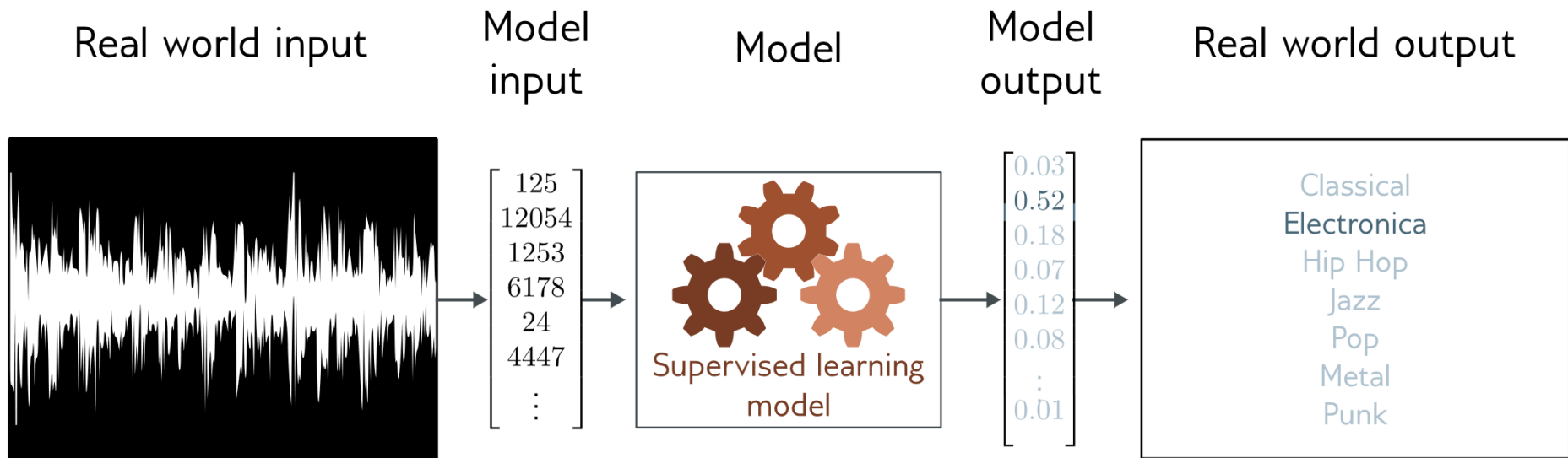
- Molecular structure
- Multivariate regression problem (>1 output, real value)
- Graph neural network

Text classification



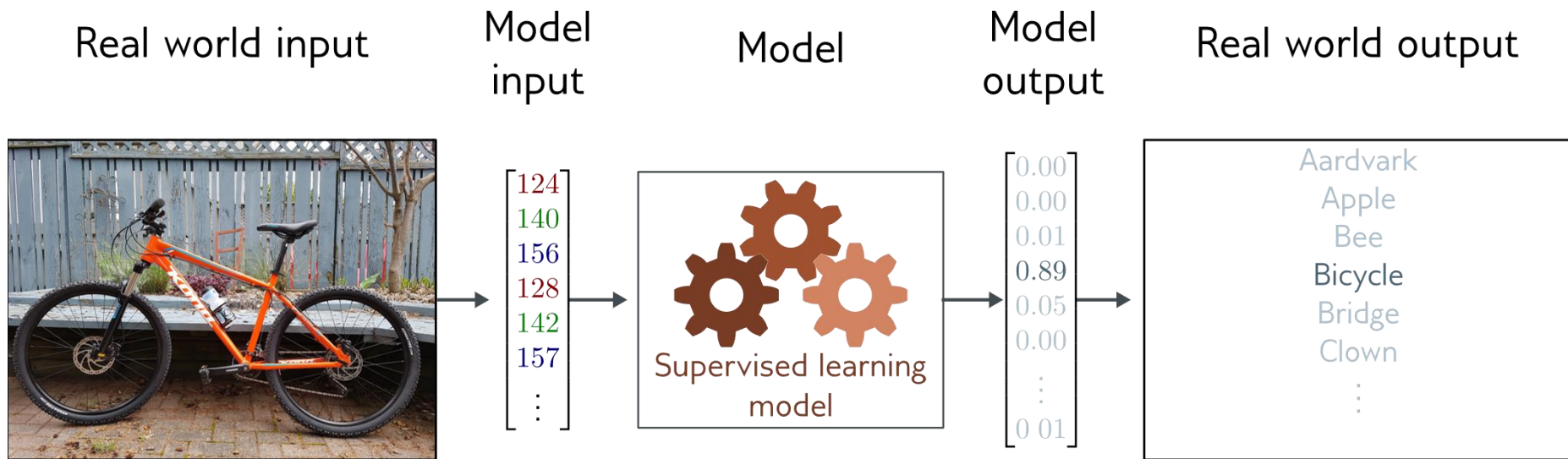
- Binary classification problem (two discrete classes)
- Transformer network

Music genre classification



- Multiclass classification problem (discrete classes, >2 possible values)
- Convolutional network

Image classification

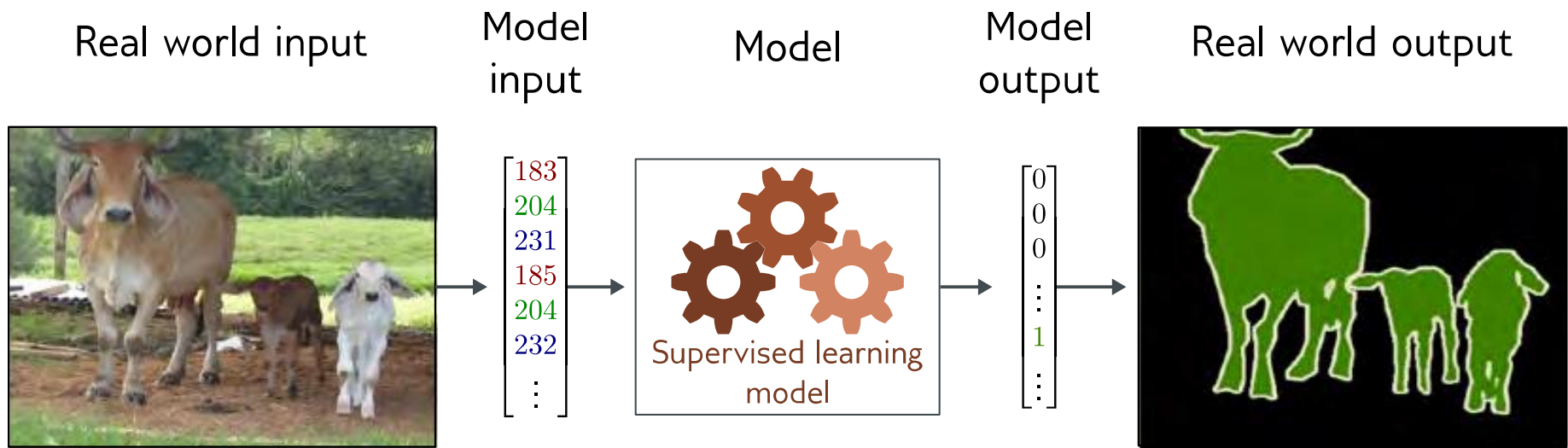


- Multiclass classification problem (discrete classes, >2 possible classes)
- Convolutional network

Five more examples

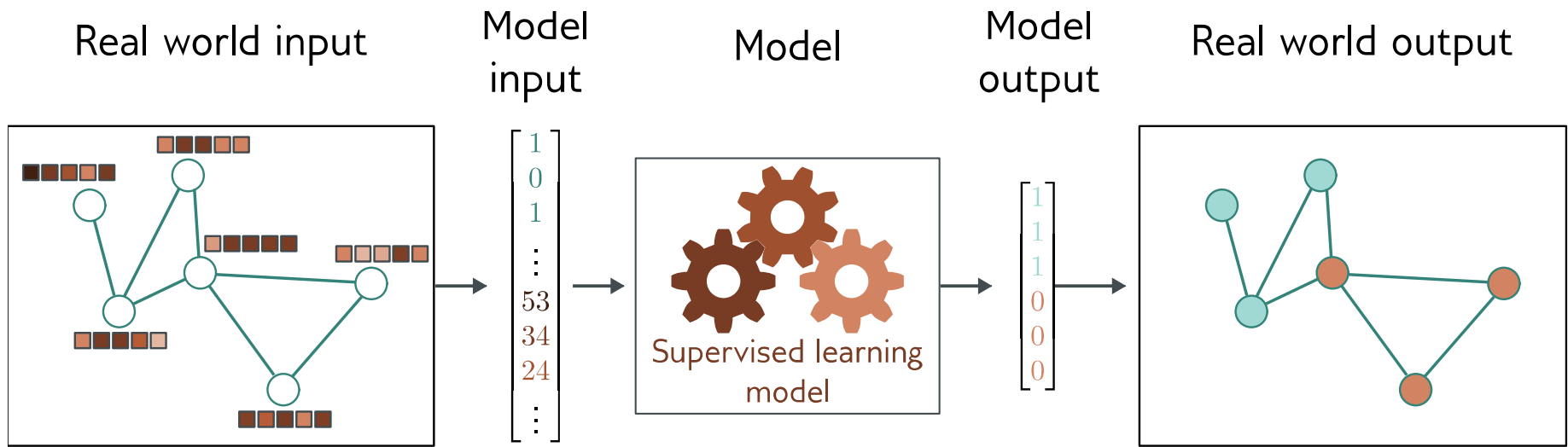
- More complex outputs
- But constrained by input (labelling)

Image segmentation



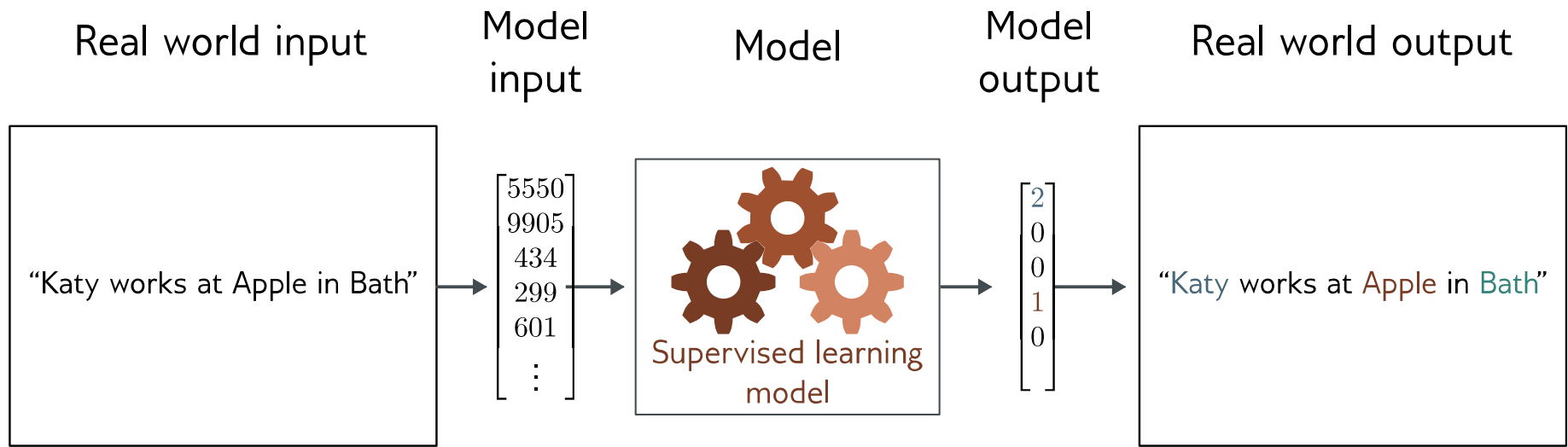
- Multivariate binary classification problem (many outputs, two discrete classes)
- Convolutional encoder-decoder network

Graph node classification



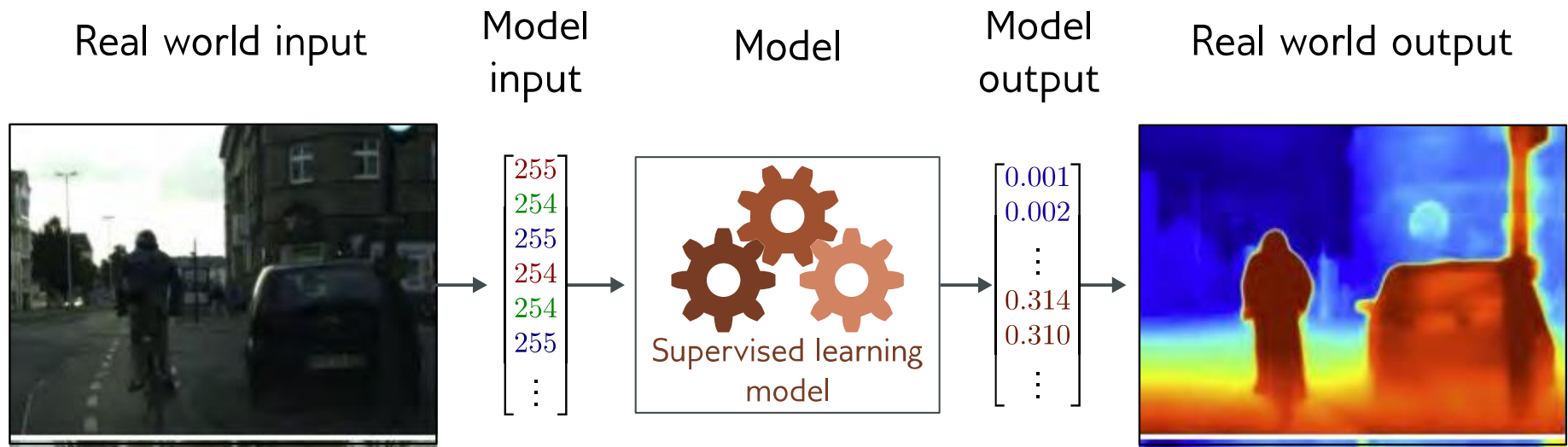
- Multivariate binary classification problem (many outputs, two discrete classes)
- Graph neural network

Entity classification



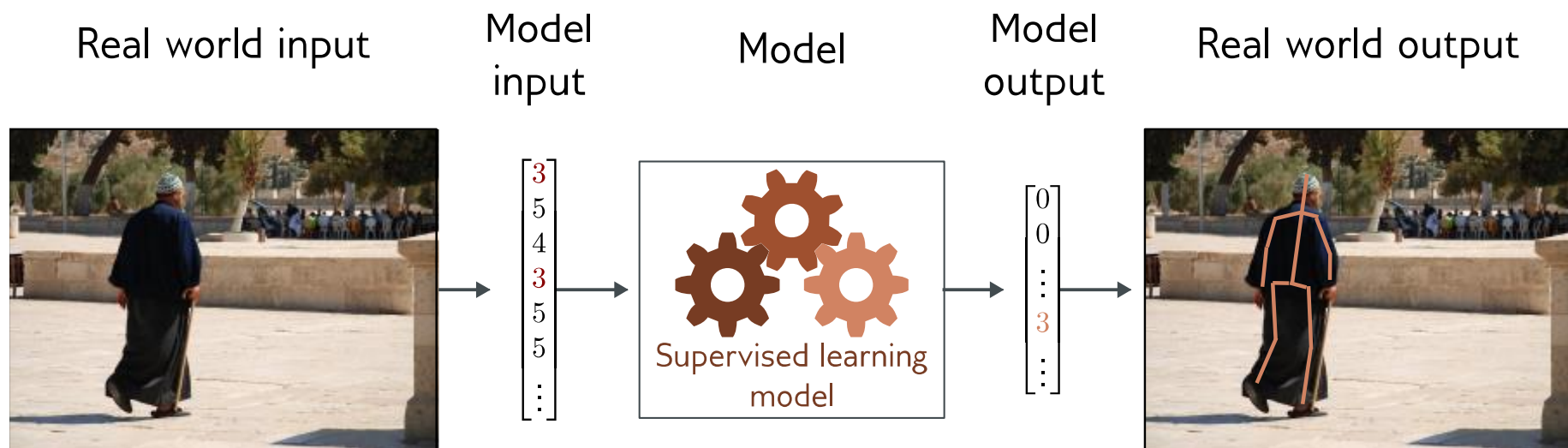
- Multivariate multiclass classification
- Transformer network

Depth estimation



- Multivariate regression problem (many outputs, continuous)
- Convolutional encoder-decoder network

Pose estimation



- Multivariate regression problem (many outputs, continuous)
- Convolutional encoder-decoder network

Terms

- Classification = discrete classes as output
- Regression = continuous numbers as output
- Two class and multiclass classification treated differently
- Univariate = one output
- Multivariate = more than one output

Three more examples

Translation

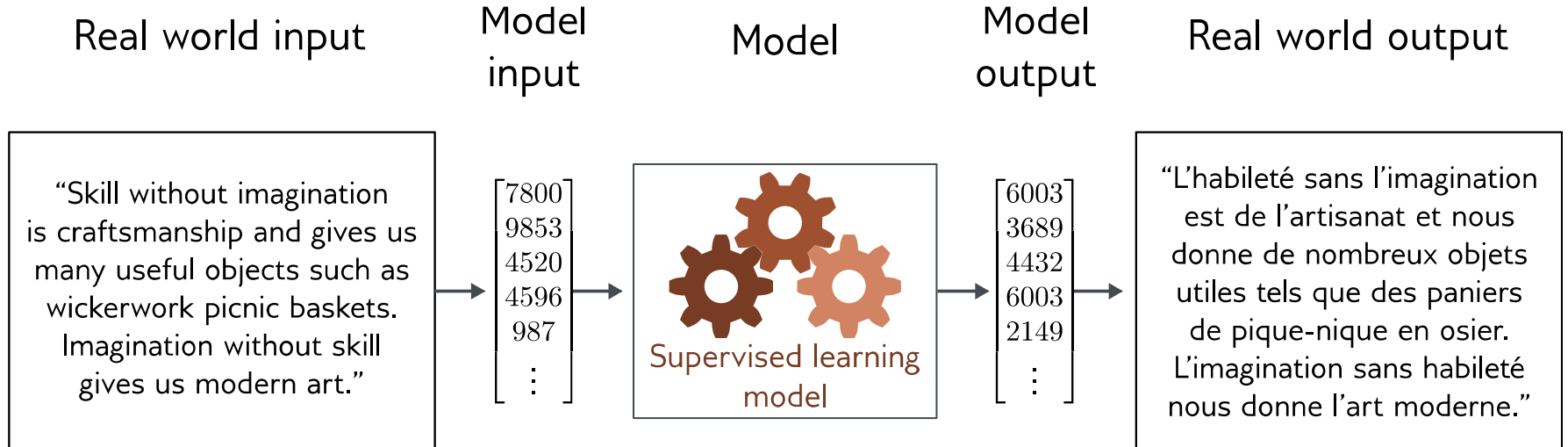


Image captioning

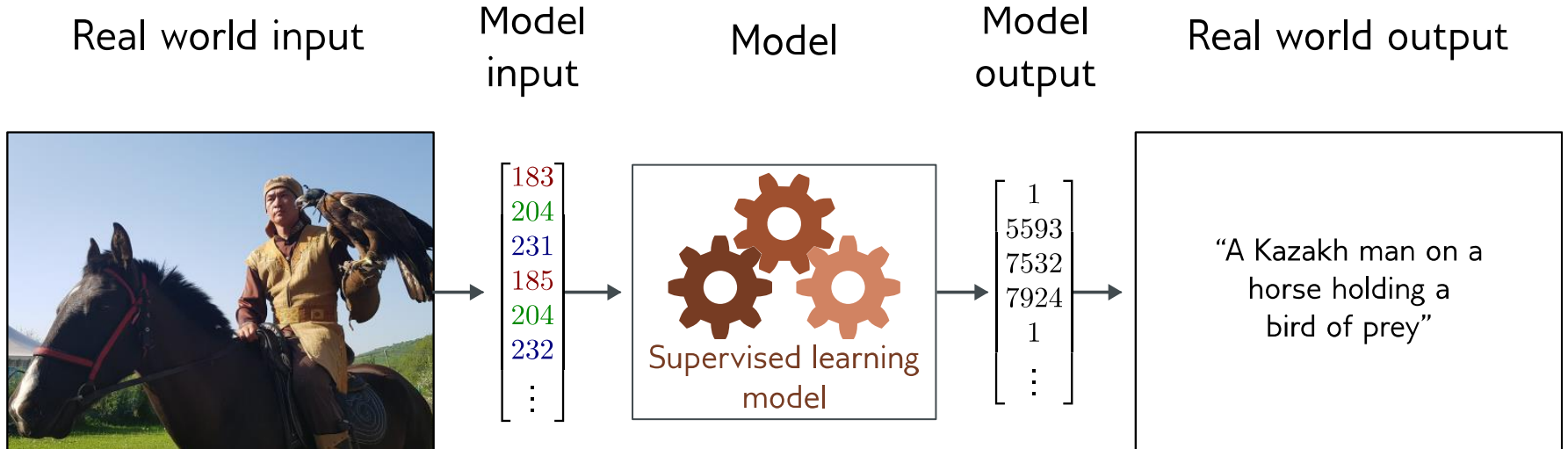
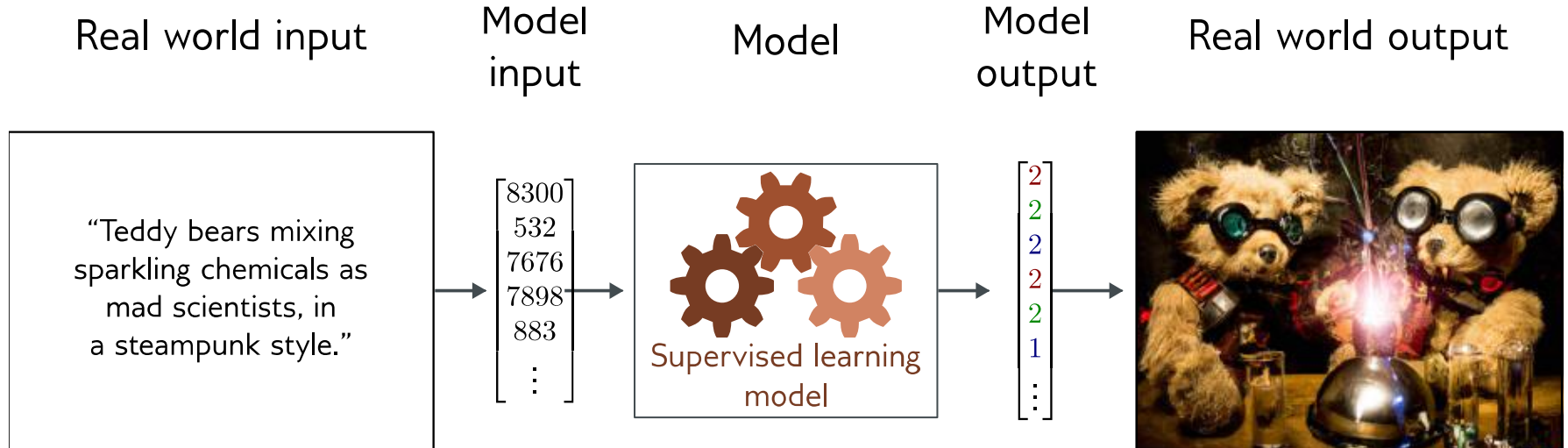


Image generation from text



What do these examples have in common?

- Very complex relationship between input and output
- Sometimes may be many possible valid answers
- But outputs (and sometimes inputs) obey rules

"A Kazakh man on a
horse holding a
bird of prey"

Language obeys
grammatical rules



Natural images also
have "rules"

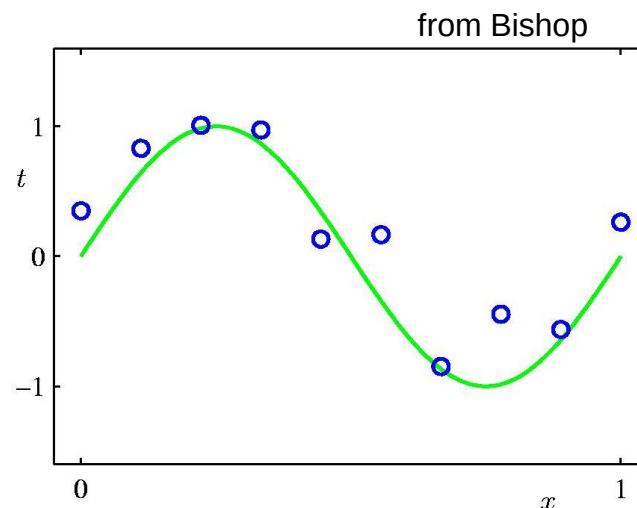
A sampling assumption

- Assume that the training examples are drawn **i**ndependently from the set of all possible examples.
- Assume that each time a training example is drawn, it comes from an **i**dentical **d**istribution (**i.i.d**)
- Assume that the test examples are drawn in exactly the same way – i.i.d. and from the same distribution as the training data.
- These assumptions make it very unlikely that a strong regularity in the training data will be absent in the test data.

A simple example: Fitting a polynomial

- The green curve $\sin(2\pi x)$ is the true function (which is not a polynomial)
- Training set: $N=10$ data points
- Input variable

$$\mathbf{x} = (x_1, x_2, \dots, x_N)^T$$



- Corresponding observations $\mathbf{t} = (t_1, t_2, \dots, t_N)^T$
- The data points are uniform in x but have noise in y .
- New value of input variable $\rightarrow$ make prediction value of the output?

A simple example: Fitting a polynomial

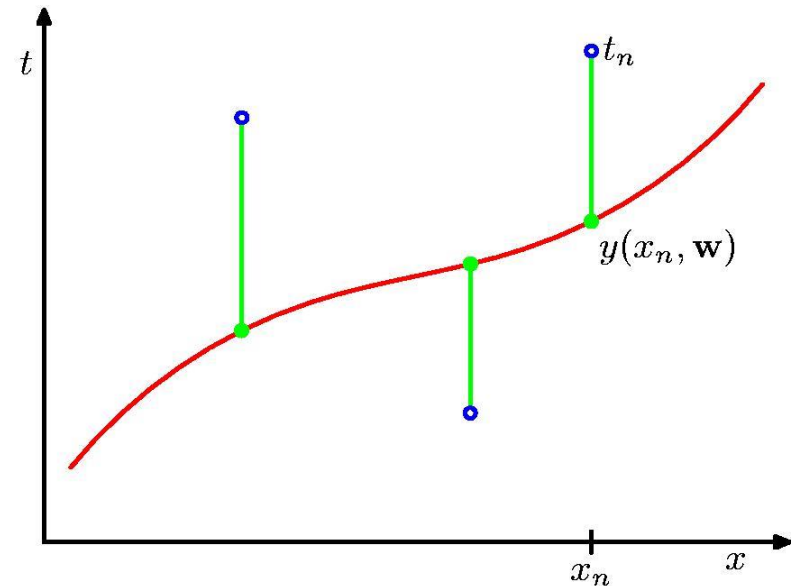
Consider a simple approach based on curve fitting

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

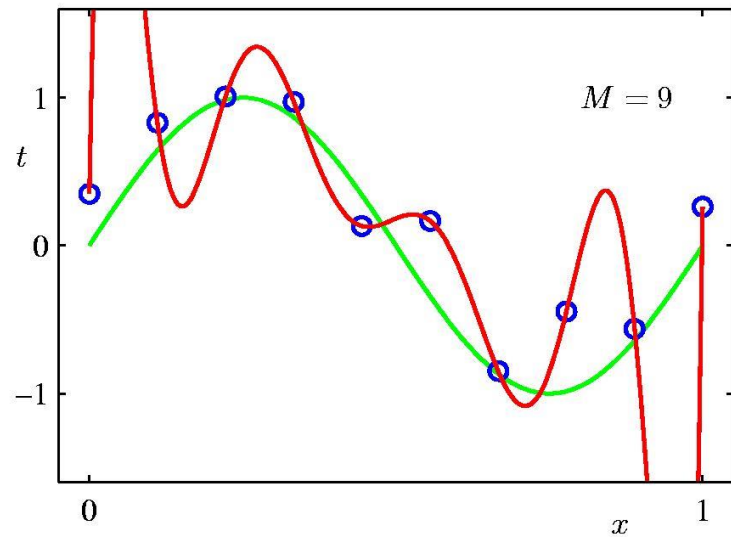
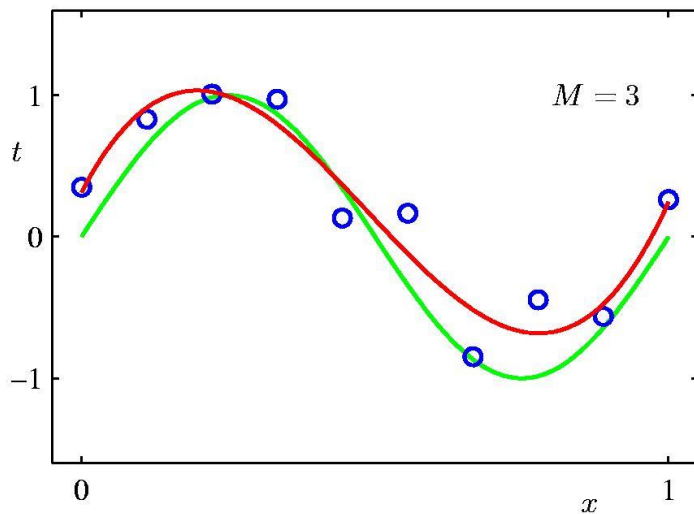
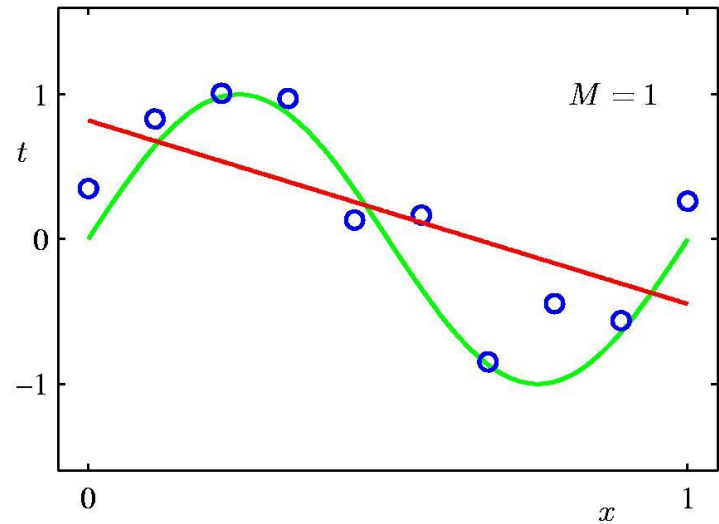
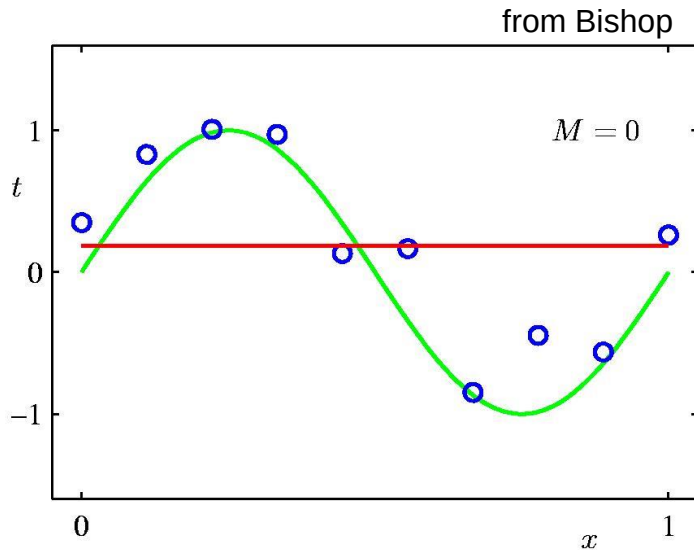
M is the *order* of the polynomial,

- We will use a loss function that measures the squared error in the prediction of $y(x)$ from x . The loss for the red polynomial is the sum of the squared vertical errors.
- So that we minimize the error function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

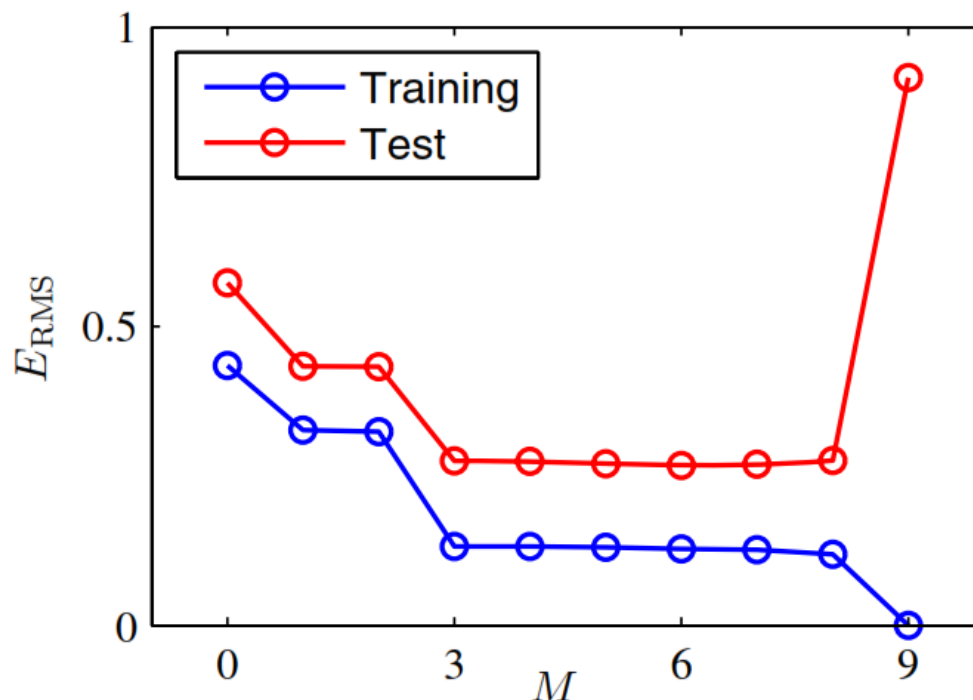


Some fits to the data: which is best?



Root mean square (RMS) error

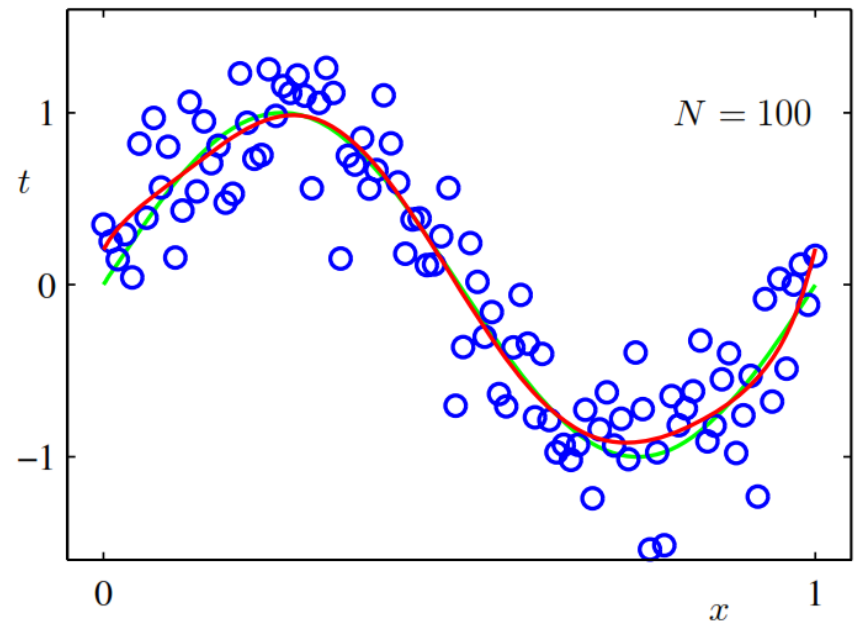
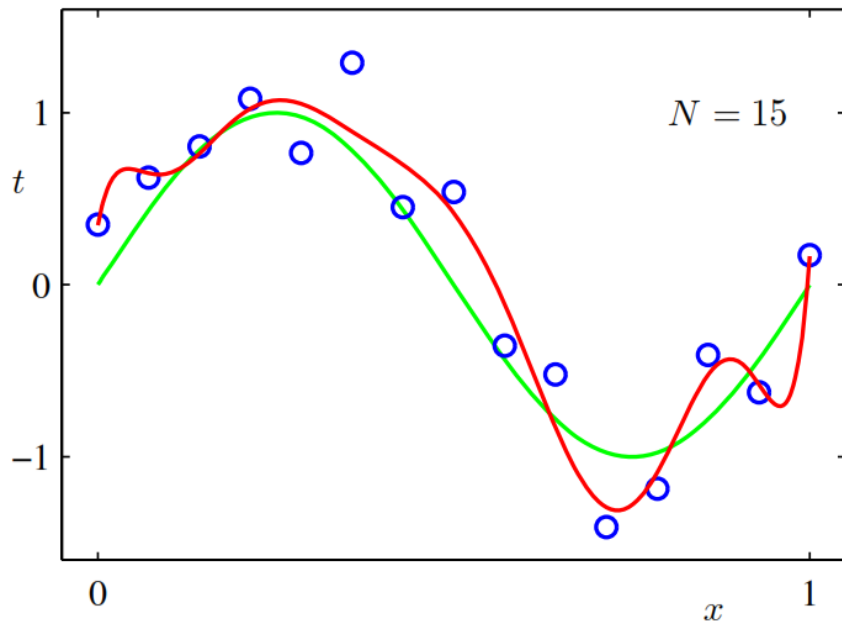
$$E_{RMS} = \sqrt{2E(w^*)/N}$$



The test set error is a measure of how well we are doing in predicting the values of t for new data observations of x

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

The values of the coefficients w^* obtained from polynomials of various order M



The sum-of-squares error function using the $M = 9$

- We see that increasing the size of the data set reduces the overfitting problem
- The larger the data set, the more complex (in other words more flexible) the model that we can afford to fit to the data

A simple way to reduce model complexity

- If we penalize polynomials that have big values for their coefficients, we will get less wiggly solutions:

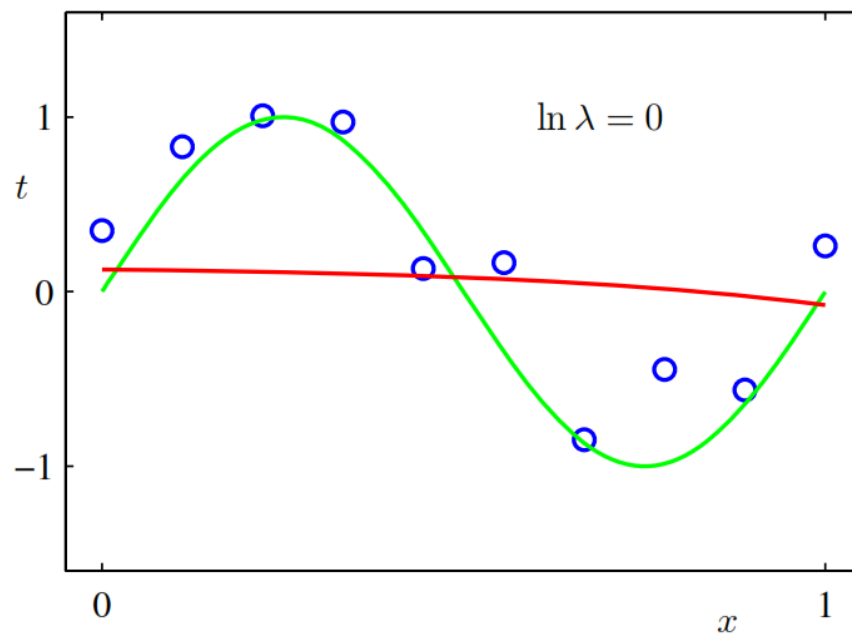
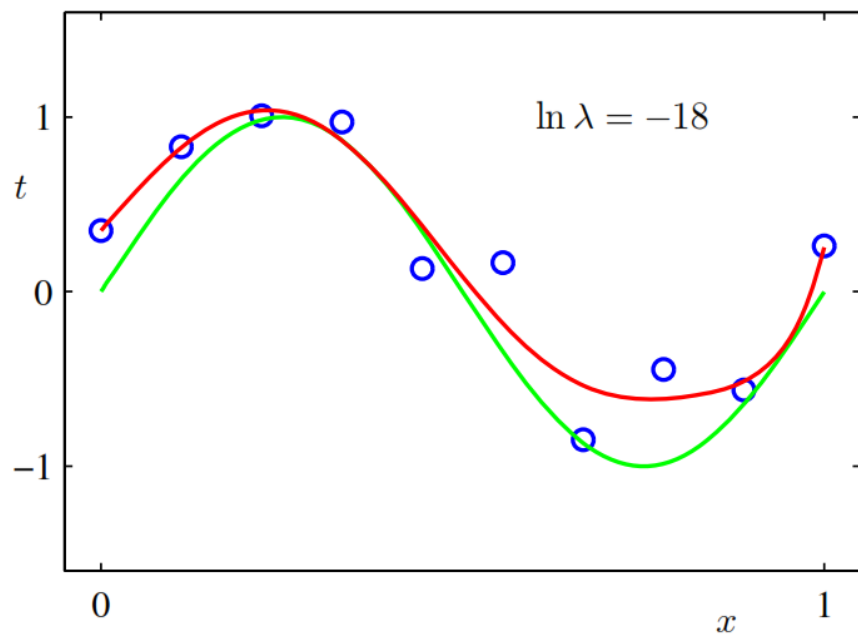
penalized loss
function

regularization
parameter

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

target value

where $\|\mathbf{w}\|^2 \equiv \mathbf{w}^T \mathbf{w} = w_0^2 + w_1^2 + \dots + w_M^2$



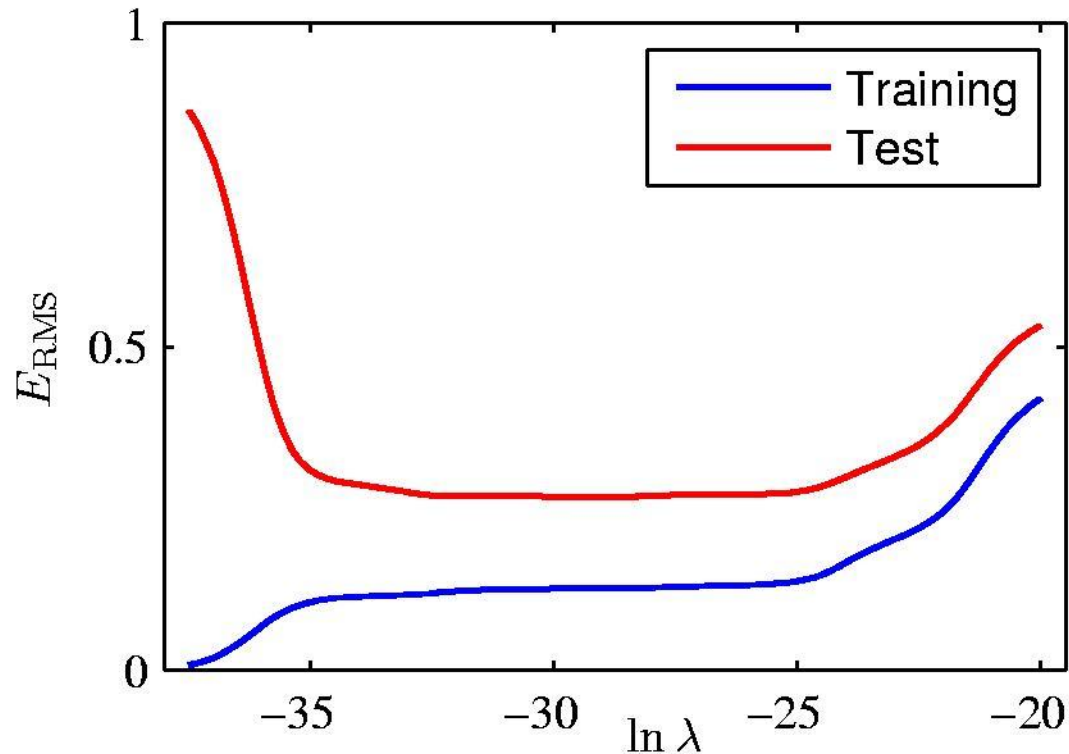
Plots of $M=9$ polynomials fitted to the data set

for a value of $\ln \lambda = -18$, the over-fitting has been suppressed
we use too large a value for λ then we again obtain a poor fit,

Polynomial Coefficients

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

Regularization: $\ln \lambda$ vs. E_{RMS}



We see that in effect λ now controls the effective complexity of the model and hence determines the degree of over-fitting

Using a validation set

- Divide the total dataset into three subsets:
 - **Training data** is used for learning the parameters of the model.
 - **Validation data** is not used of learning but is used for deciding what type of model and what amount of regularization works best.
 - **Test data** is used to get a final, unbiased estimate of how well the network works. We expect this estimate to be worse than on the validation data.
- We could then re-divide the total dataset to get another unbiased estimate of the true error rate.

Some Loss Functions

- Squared difference between actual and target real-valued outputs.
- Number of classification errors
 - Problematic for optimization because the derivative is not smooth.
- Negative log probability assigned to the correct answer.
 - This is usually the right function to use.
 - In some cases it is the same as squared error (regression with Gaussian output noise)
 - In other cases it is very different (classification with discrete classes needs cross-entropy error)

Generalization

- The real aim of supervised learning is to do well on test data that is not known during learning.
- Choosing the values for the parameters that minimize the loss function on the training data is not necessarily the best policy.
- We want the learning machine to model the true regularities in the data and to ignore the noise in the data.
 - But the learning machine does not know which regularities are real and which are accidental quirks of the particular set of training examples we happen to pick.
- So how can we be sure that the machine will generalize correctly to new data?

Trading off the goodness of fit against the complexity of the model

- It is intuitively obvious that you can only expect a model to generalize well if it explains the data surprisingly well given the complexity of the model.
- If the model has as many degrees of freedom as the data, it can fit the data perfectly but so what?
- There is a lot of theory about how to measure the model complexity and how to control it to optimize generalization.
 - Some of this “learning theory” will be covered later in the course, but it requires a whole course on learning theory to cover it properly.